Austin Duckworth
CSCI580, Artificial Intelligence, Fall2026
Assignment 4 Report

The first task was to implement the acceptance rule and cooling schedule for the provided Simulated Annealing algorithm. This was very simple, as the formulas were provided in the instructions- we only needed to translate them to python code.

My solutions to these tasks were

```
# TODO: acceptance rule
# Is accept True or false when delta <= 0 or delta >0?
if delta <= 0 or (rng.random() < math.e**(-1 * delta / T)):
    accept = True
```

and

```
# TODO: cooling schedule
# How is T adjusted?
T = max(T * cfg.alpha, 1 * 10**(-12))
```

respectively.

The acceptance rule was to be True always if delta was less than or equal to 0, or with probability exp(-delta/T) if delta was bigger than zero, and False otherwise. I encapsulated both True scenarios in a combined if-statement, and since accept defaulted to False, no else statement was necessary.

The cooling schedule was even easier. I simply combined the two requirements, T = T * alpha at each iteration and T = max(T, 1e-12) into a single statement, T = max(T * alpha, 1e-12) and converted that into python code.

(More on Next Page for image formatting reasons)

The second task was to implement the selection, crossover, and mutation steps in the provided genetic algorithm skeleton. This was much more difficult.

My solutions to these tasks were

```
# TODO: tournament selection (minimize length)
tempList = [rng.randint(0, len(pop) - 1) for _ in range(k)]
fittest = tempList[0]
for i in tempList:
    if lengths[i] < lengths[fittest]:
        fittest = i

return pop[fittest]
```

,

```
# TODO: order crossover (OX)
cut1 = rng.randint(0, len(parent1) - 1)
cut2 = rng.randint(0, len(parent1) - 1)
while (cut1 == cut2):
    cut2 = rng.randint(0, len(parent1) - 1)
if cut1 > cut2:
    cut1, cut2 = cut2, cut1

tempList = parent1[:]
Slice = parent2[cut1:cut2+1]

for e in Slice:
    if e in tempList:
        tempList.remove(e)

child = tempList[:cut1] + Slice + tempList[cut1:]
return child
```

,

and

```
# TODO: swap mutation
return random_two_opt_neighbor(tour, rng)
```

,

respectively.

The mutation step was the easiest, as the method of doing mutation was provided, so all that was needed was to return the result of that method.

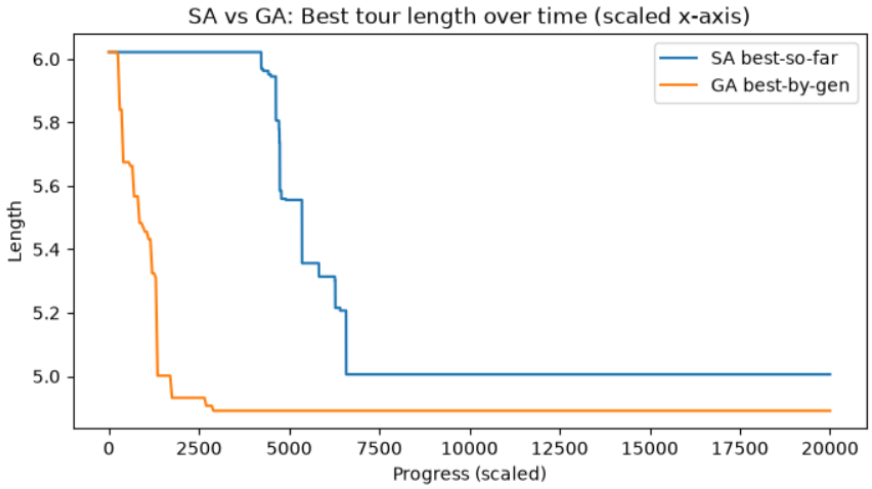
The selection step had medium difficulty. It involved selecting k random indices corresponding to k random tours, and select the shortest tour among them to return. I did this with a simple for loop.

The crossover step was the most difficult for me. It required passing down the ‘genes’ of two parent tours to a new child tour while preserving what makes the tours short in the first place (their order). I did this by taking a slice of random length > 0 from parent2, and removing each element in that slice from parent1. Then, the new child would consist of that slice of parent2 in the same position it was at in parent2, with the rest of the values filled in with the same order they were at in parent1. I did this by iterating through the Slice and removing each element from a temporary list (to avoid changing parent1 in case it got selected again before the next generation phased it out). I didn’t iterate through the temporary list to avoid changing and iterating through the same list at the same time. Once that was done, I simply combined the new temporary list which was a copy of parent1 without any values found in the slice, up to the point where the slice was, combined it with the slice, and then continued on with the rest of the values from the temporary list.

A big challenge I had in this section was that I kept accidentally leaving out or skipping over one element when creating children, which led to errors and invalid tours as the program ran.

Results / Comparison
Both my Simulated Annealing and Genetic Algorithm out-performed the Nearest-Neighbor algorithm, and my Genetic Algorithm out-performed my Simulated Annealing algorithm.

Below is the comparison between their best solution over time. Notice how the Genetic Algorithm not only reaches the shortest path overall, but also that it consistently reaches shorter paths faster.



We can see that Simulated Annealing has a long period of little-to-no progress before it starts rapidly approaching a good solution, whereas my Genetic Algorithm started approaching a good solution right away, and found a better solution overall. In the end, my genetic algorithm reached its best solution a little over twice as fast as the Simulated Annealing algorithm.

My genetic algorithm seemed to be more consistent, accepting slightly better solutions more often, whereas the Simulated Annealing algorithm appears to have more variability, with long periods of stagnation before suddenly finding a much better solution.

Both Simulated Annealing and my Genetic Algorithm seemed to be capable of escaping local minima, indicated by the large vertical jumps showing a drastic improvement in path length. Simulated Annealing lost that ability as time went on, however, due to the cooling schedule, which meant it ultimately ended on a worse solution than my Genetic Algorithm.

My Genetic Algorithm definitely lost diversity as it progressed. After the algorithm was done, I included this snippet of code to test how many copies of the best solution were in the population compared to the population size.

```
sameCount = 0
for i in range(len(pop)):
    if pop[i] == best_tour:
        sameCount += 1
print(f'sameCount:{sameCount}')
print(len(pop))
```

The answer I got was 170 / 200 tours in the population were the best solution, meaning 85% of the population was the best solution. This lack of diversity is natural, as the point of the genetic algorithm is for a population of random solutions to converge towards the best solution, and naturally most solutions that are close the best are also similar to or the same as each other.

In Conclusion

Both Simulated Annealing and Genetic Algorithms are useful for finding the best solution. They are both far better than a Nearest-Neighbor Algorithm (My SA algorithm outperformed it by 16.87%, and my Genetic Algorithm by 18.77%). However, Genetic Algorithms seem to start approaching the best solution faster, and reach better solutions overall. There was not one time during testing when my genetic algorithm was outperformed by simulated annealing, even when I hadn't yet implemented the crossover function (it just returned the first parent).